

HBAExpress

Gestion des membres d'un vendeur Marketplace

Seller Service - Employés, invitations, rôles, permissions, sécurité et audit

.NET 9 - PostgreSQL - gRPC - Kafka - Redis - RBAC/ABAC - Clean Architecture

Objectif : fournir une spécification suffisamment précise pour permettre à Claude Code de générer le module de gestion des membres du Seller Service et son intégration avec Catalog, Inventory, Marketplace Order, Review, Return/Refund, Wallet et les BFF.

1. Objectif métier

Un vendeur HBAExpress peut exploiter une ou plusieurs boutiques et déléguer certaines opérations à des employés. Chaque employé possède son propre compte utilisateur. Aucun partage du compte propriétaire n'est autorisé. Les accès sont accordés au niveau de l'organisation vendeur et/ou d'une boutique, avec rôles et permissions explicites.

```
SELLER / ORGANISATION
|
+-- Owner
|
+-- Store A
|   +-- Store Admin
|   +-- Order Manager
|   +-- Inventory Manager
|   +-- Catalog Manager
|   +-- Customer Support
|
+-- Store B
|   +-- Store Manager
|   +-- Employee
```

2. Séparation des concepts

Concept	Responsabilité
USER	Personne qui s'authentifie. Géré par Identity/User Service.
SELLER	Organisation commerciale / compte vendeur.
STORE	Boutique exploitée par un Seller.
SELLER_MEMBER	Lien entre un User et l'organisation Seller.
STORE_MEMBER	Affectation d'un membre à une boutique.
ROLE	Regroupement nommé de permissions.
PERMISSION	Action atomique autorisable.
INVITATION	Invitation temporaire permettant de rejoindre l'organisation/boutique.

Ne pas ajouter simplement un booléen `isEmployee` dans User. Le compte utilisateur doit rester indépendant de son appartenance commerciale.

3. Source de vérité

Identity Service reste source de vérité de l'identité et de l'authentification. Seller Service devient source de vérité de l'appartenance vendeur, des boutiques, des rôles et des permissions Marketplace.

```
Identity Service
|
|  userId
|  v
Seller Service
|
+-- Seller
+-- Store
+-- SellerMember
+-- StoreMembership
+-- Role
+-- Permission
+-- Invitation
+-- Audit

Catalog / Inventory / Order / Review / Return
|
+-- utilisent le contexte d'accès Seller/Store
```

4. Niveaux d'accès

Le système doit supporter deux scopes. SELLER scope donne accès à l'ensemble de l'organisation. STORE scope limite l'accès à une ou plusieurs boutiques.

```
scope = SELLER
-> ex: OWNER, FINANCE_ADMIN

scope = STORE
-> ex: ORDER_MANAGER sur store_001
-> ex: INVENTORY_MANAGER sur store_002
```

5. Statuts d'un membre

```
INVITED
ACTIVE
SUSPENDED
```

REVOKED
LEFT

SUSPENDED conserve l'historique et les affectations mais interdit immédiatement l'accès. REVOKED représente une révocation administrative. LEFT peut représenter un départ volontaire.

6. Modèle SellerMember

```
{
  "id": "sm_001",
  "sellerId": "seller_001",
  "userId": "user_123",
  "status": "ACTIVE",

  "profile": {
    "displayName": "David K.",
    "jobTitle": "Responsable commandes"
  },

  "sellerRoles": [],
  "storeMemberships": [
    {
      "storeId": "store_001",
      "roleIds": ["role_order_manager"],
      "status": "ACTIVE"
    }
  ],

  "invitedByUserId": "user_owner_001",
  "joinedAt": "2026-08-19T08:00:00Z",
  "createdAt": "2026-08-18T20:00:00Z",
  "updatedAt": "2026-08-19T08:00:00Z",
  "version": 3
}
```

7. Modèle Invitation

```
{
  "id": "invite_001",
  "sellerId": "seller_001",
  "email": "employee@example.com",
  "status": "PENDING",
  "storeAssignments": [
    {
      "storeId": "store_001",
      "roleIds": ["role_order_manager"]
    }
  ],
  "tokenHash": "...",
  "expiresAt": "2026-08-26T08:00:00Z",
  "invitedByUserId": "owner_001",
  "acceptedByUserId": null,
  "createdAt": "2026-08-19T08:00:00Z"
}
```

Le token brut d'invitation ne doit pas être stocké en base. Stocker uniquement son hash. L'invitation doit avoir une expiration et être à usage unique.

8. Statuts Invitation

PENDING
ACCEPTED
DECLINED
EXPIRED
REVOKED

9. Workflow d'invitation

```
OWNER / autorisé
|
| Invite(email, stores, roles)
v
Invitation PENDING
|
+--> Notification/Email
|
Employé ouvre le lien
|
+--> compte existe ? login
|
+--> sinon création compte Identity
|
v
AcceptInvitation
|
+--> validation token + email + expiration
+--> création SellerMember
+--> création StoreMembership(s)
+--> Invitation ACCEPTED
+--> Kafka seller.member.joined
```

10. Rôles système recommandés

Rôle	Scope	Description
OWNER	SELLER	Propriétaire. Contrôle complet, non supprimable par un employé.
SELLER_ADMIN	SELLER	Administration générale hors actions réservées Owner.
STORE_ADMIN	STORE	Administration complète d'une boutique.
STORE_MANAGER	STORE	Opérations quotidiennes.
CATALOG_MANAGER	STORE	Produits, variantes, soumission admin.
INVENTORY_MANAGER	STORE	Stocks, ajustements, transferts.
ORDER_MANAGER	STORE	Commandes et préparation.
CUSTOMER_SUPPORT	STORE	Lecture commandes, retours, réponses client selon droits.
FINANCE_MANAGER	SELLER/STORE	Finances en lecture/gestion selon politique.
EMPLOYEE	STORE	Rôle minimal personnalisable.

11. Permissions Catalog

PRODUCT_VIEW
PRODUCT_CREATE
PRODUCT_UPDATE
PRODUCT_ARCHIVE
PRODUCT_SUBMIT_FOR_REVIEW
PRODUCT_PUBLISH
PRODUCT_UNPUBLISH
PRODUCT_MEDIA_MANAGE
CATEGORY_VIEW
BRAND_VIEW

PRODUCT_PUBLISH ne contourne jamais la validation admin du Catalog Service. Il autorise seulement le membre à publier un produit déjà APPROVED.

12. Permissions Inventory

INVENTORY_VIEW
INVENTORY_INITIALIZE
INVENTORY_ADJUST
INVENTORY_TRANSFER
INVENTORY_THRESHOLD_UPDATE
STOCK_MOVEMENT_VIEW
STOCK_LOCATION_VIEW
STOCK_LOCATION_MANAGE

13. Permissions Order

ORDER_VIEW
ORDER_CONFIRM
ORDER_REJECT
ORDER_MARK_PREPARING
ORDER_MARK_READY
ORDER_CANCEL
ORDER_EXPORT
ORDER_CUSTOMER_CONTACT_VIEW

14. Permissions Review / Return

REVIEW_VIEW
REVIEW_REPLY

RETURN_VIEW
RETURN_APPROVE
RETURN_REJECT
RETURN_CONFIRM_RECEIVED
RETURN_INSPECT
RETURN_DISPUTE_VIEW

15. Permissions membres et sécurité

MEMBER_VIEW
MEMBER_INVITE
MEMBER_SUSPEND
MEMBER_REVOKE
MEMBER_ASSIGN_STORE
MEMBER_ASSIGN_ROLE

ROLE_VIEW
ROLE_CREATE
ROLE_UPDATE
ROLE_DELETE
ROLE_ASSIGN

AUDIT_VIEW

16. Permissions financières sensibles

FINANCE_VIEW
WALLET_VIEW
TRANSACTION_VIEW
PAYOUT_VIEW
PAYOUT_CONFIGURE
WITHDRAWAL_REQUEST
BANK_ACCOUNT_VIEW
BANK_ACCOUNT_UPDATE

BANK_ACCOUNT_UPDATE, PAYOUT_CONFIGURE et WITHDRAWAL_REQUEST doivent être considérées comme hautement sensibles. Elles peuvent être réservées à OWNER, protégées par réauthentification/MFA et soumises à audit renforcé.

17. Permissions Owner-only recommandées

SELLER_CLOSE
SELLER_OWNERSHIP_TRANSFER
BANK_ACCOUNT_UPDATE
PAYOUT_CONFIGURE
OWNER_MANAGE
SECURITY_POLICY_UPDATE

18. Rôles personnalisés

Le vendeur peut créer des rôles personnalisés à partir des permissions autorisées. Un rôle personnalisé ne peut jamais s'octroyer une permission que l'acteur créateur ne possède pas ou une permission marquée Owner-only.

```
{
  "name": "Préparateur commandes",
  "scope": "STORE",
  "permissions": [
    "ORDER_VIEW",
    "ORDER_MARK_PREPARING",
    "ORDER_MARK_READY",
    "INVENTORY_VIEW"
  ]
}
```

19. Matrice d'autorisation

Action	Owner	Store Admin	Order Mgr	Inventory Mgr	Catalog Mgr
Voir produits	Oui	Oui	Lecture	Lecture	Oui
Créer/modifier produit	Oui	Oui	Non	Non	Oui
Ajuster stock	Oui	Oui	Non	Oui	Non
Confirmer commande	Oui	Oui	Oui	Non	Non
Marquer prête	Oui	Oui	Oui	Non	Non
Inviter membre	Oui	Config.	Non	Non	Non
Modifier banque	Oui	Non	Non	Non	Non

20. PostgreSQL - tables

```
seller_members
store_memberships
seller_invitations

roles
permissions
role_permissions
seller_member_roles
store_membership_roles

member_permission_overrides  (optionnel)
seller_security_policies

seller_member_audit_logs

outbox_events
consumer_inbox
```

21. seller_members

```
id UUID PRIMARY KEY
seller_id UUID NOT NULL
user_id UUID NOT NULL
status VARCHAR(30) NOT NULL

display_name VARCHAR(150) NULL
job_title VARCHAR(120) NULL

invited_by_user_id UUID NULL
joined_at TIMESTAMPTZ NULL

created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
version BIGINT NOT NULL DEFAULT 0

UNIQUE(seller_id, user_id)
```

22. store_memberships

```
id UUID PRIMARY KEY
seller_member_id UUID NOT NULL
store_id UUID NOT NULL
status VARCHAR(30) NOT NULL

created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL

UNIQUE(seller_member_id, store_id)
```

23. roles / permissions

```

roles
-----
id UUID PK
seller_id UUID NULL
name VARCHAR(100)
scope VARCHAR(20) -- SELLER | STORE
is_system_role BOOLEAN
is_owner_only BOOLEAN
created_at TIMESTAMPTZ

permissions
-----
id UUID PK
code VARCHAR(120) UNIQUE
domain VARCHAR(50)
description VARCHAR(255)
risk_level VARCHAR(20) -- NORMAL | SENSITIVE | CRITICAL

role_permissions
-----
role_id UUID
permission_id UUID
PRIMARY KEY(role_id, permission_id)

```

24. API - membres

Méthode	Route	Usage
GET	/api/v1/seller/members	Lister les membres
GET	/api/v1/seller/members/{id}	Détail d'un membre
POST	/api/v1/seller/members/invitations	Inviter
GET	/api/v1/seller/members/invitations	Lister invitations
POST	/api/v1/seller/invitations/{token}/accept	Accepter
POST	/api/v1/seller/invitations/{id}/resend	Renvoyer
DELETE	/api/v1/seller/invitations/{id}	Révoquer invitation
POST	/api/v1/seller/members/{id}/suspend	Suspendre
POST	/api/v1/seller/members/{id}/activate	Réactiver
DELETE	/api/v1/seller/members/{id}	Révoquer accès

25. API - affectations et rôles

Méthode	Route	Usage
PUT	/api/v1/seller/members/{id}/stores	Modifier affectations boutiques
PUT	/api/v1/seller/members/{id}/roles	Modifier rôles Seller
PUT	/api/v1/seller/members/{id}/stores/{storeId}/roles	Rôles boutique
GET	/api/v1/seller/roles	Lister rôles
POST	/api/v1/seller/roles	Créer rôle personnalisé
PATCH	/api/v1/seller/roles/{id}	Modifier rôle
DELETE	/api/v1/seller/roles/{id}	Supprimer rôle
GET	/api/v1/seller/permissions	Catalogue permissions

26. Exemple invitation

```

POST /api/v1/seller/members/invitations

{
  "email": "employee@example.com",
  "jobTitle": "Responsable commandes",
  "assignments": [
    {
      "storeId": "store_001",
      "roleIds": ["role_order_manager"]
    }
  ]
}

{
  "success": true,
  "data": {
    "invitationId": "invite_001",
    "email": "employee@example.com",

```

```

    "status": "PENDING",
    "expiresAt": "2026-08-26T08:00:00Z"
  }
}

```

27. Exemple modification des rôles

PUT /api/v1/seller/members/sm_001/stores/store_001/roles

```

{
  "roleIds": [
    "role_order_manager",
    "role_customer_support"
  ]
}

```

28. Réponse détail membre

```

{
  "success": true,
  "data": {
    "id": "sm_001",
    "userId": "user_123",
    "status": "ACTIVE",
    "profile": {
      "displayName": "David K.",
      "email": "employee@example.com",
      "jobTitle": "Responsable commandes"
    },
    "stores": [
      {
        "storeId": "store_001",
        "storeName": "HBA Tech Store",
        "roles": ["ORDER_MANAGER"],
        "effectivePermissions": [
          "ORDER_VIEW",
          "ORDER_CONFIRM",
          "ORDER_MARK_PREPARING",
          "ORDER_MARK_READY"
        ]
      }
    ]
  },
  "joinedAt": "2026-08-19T08:00:00Z"
}

```

29. Résolution d'autorisation

Authorize(userId, sellerId, storeId, permission)

1. vérifier identité authentifiée
2. charger SellerMember
3. status doit être ACTIVE
4. vérifier StoreMembership si scope STORE
5. résoudre rôles actifs
6. agréger permissions
7. appliquer restrictions Owner-only
8. appliquer overrides/policies éventuels
9. allowed = permission présente
10. journaliser les actions sensibles

30. gRPC SellerAccessService

```

service SellerAccessService {
  rpc GetMemberAccess(GetMemberAccessRequest)
    returns (MemberAccessResponse);

  rpc HasPermission(HasPermissionRequest)
    returns (HasPermissionResponse);

  rpc GetBatchPermissions(GetBatchPermissionsRequest)
    returns (GetBatchPermissionsResponse);
}

message HasPermissionRequest {
  string user_id = 1;
  string seller_id = 2;
  string store_id = 3;
  string permission = 4;
}

message HasPermissionResponse {
  bool allowed = 1;
  string member_id = 2;
  repeated string roles = 3;
}

```

31. JWT et cache

Éviter un appel gRPC Seller Service pour chaque requête. Les claims stables peuvent être présents dans le token, mais les permissions fines doivent pouvoir être invalidées rapidement. Recommandation : contexte minimal dans JWT + cache Redis court des permissions effectives + invalidation Kafka.

```
JWT:
  sub=user_123
  session_id=...
  tenant hints (optionnels)

Redis:
  seller-access:{sellerId}:{storeId}:{userId}
  TTL: court, ex. 2-5 minutes

Kafka:
  seller.member.permissions-updated
  -> invalidate cache immédiatement
```

Ne pas placer une liste énorme de permissions et toutes les boutiques dans un JWT longue durée : les droits révoqués resteraient utilisables jusqu'à expiration.

32. Événements Kafka

- seller.member.invited
- seller.member.invitation-accepted
- seller.member.joined
- seller.member.suspended
- seller.member.activated
- seller.member.revoked
- seller.member.store-assigned
- seller.member.store-unassigned
- seller.member.roles-updated
- seller.member.permissions-updated
- seller.role.created
- seller.role.updated
- seller.role.deleted

33. Structure Kafka standard

```
{
  "eventId": "0198...",
  "eventType": "seller.member.permissions-updated",
  "eventVersion": 1,
  "occurredAt": "2026-08-19T09:00:00Z",
  "producer": "seller-service",
  "environment": "production",
  "correlationId": "req_001",
  "causationId": "cmd_001",
  "traceId": "4bf92f...",
  "aggregate": {
    "type": "SellerMember",
    "id": "sm_001",
    "version": 4
  },
  "partitionKey": "seller_001",
  "data": {
    "sellerId": "seller_001",
    "memberId": "sm_001",
    "userId": "user_123",
    "affectedStoreIds": ["store_001"]
  }
}
```

34. Audit

```
{
  "id": "audit_001",
  "sellerId": "seller_001",
  "storeId": "store_001",

  "actor": {
    "userId": "user_123",
    "memberId": "sm_001"
  },

  "action": "ORDER_STATUS_CHANGED",
```

```

"resource": {
  "type": "SELLER_ORDER",
  "id": "sorder_123"
},

"changes": {
  "from": "CONFIRMED",
  "to": "PREPARING"
},

"ipAddress": "masked-or-controlled",
"occurredAt": "2026-08-19T09:30:00Z",
"traceId": "4bf92f..."
}

```

35. Actions à auditer obligatoirement

- invitation/révocation d'un membre
- modification des rôles/permissions
- modification d'informations financières
- retrait/payout
- ajustement de stock
- changement de statut commande
- approbation/rejet d'un retour
- publication/dépublication produit
- modification des paramètres de sécurité

36. Protection contre l'escalade de privilèges

- un membre ne peut attribuer que les permissions qu'il est autorisé à administrer
- un STORE_ADMIN ne peut pas se donner des droits Seller/Owner
- un utilisateur ne peut pas modifier ses propres droits critiques si la politique l'interdit
- le dernier OWNER ne peut pas être supprimé/révoqué sans transfert de propriété
- une affectation storeId doit appartenir au même sellerId
- sellerId/storeId provenant du body ne constitue jamais une preuve d'autorisation
- les endpoints sensibles doivent recalculer les droits côté serveur

37. Sécurité des actions critiques

Pour les actions financières ou de sécurité critiques, prévoir une politique de step-up authentication : MFA ou réauthentification récente, notification de sécurité et audit renforcé.

Exemples:
 BANK_ACCOUNT_UPDATE
 PAYOUT_CONFIGURE
 WITHDRAWAL_REQUEST
 SELLER_OWNERSHIP_TRANSFER
 SECURITY_POLICY_UPDATE

38. Intégration Catalog

Employé -> Catalog API/BFF
 Catalog reçoit:
 authenticated userId
 sellerId/storeId ciblé

Catalog:
 -> autorisation locale via access context/cache
 ou SellerAccess gRPC
 -> Require PRODUCT_CREATE

Lors de SubmitForReview:
 -> Require PRODUCT_SUBMIT_FOR_REVIEW

Lors de Publish:
 -> produit doit être APPROVED
 -> Require PRODUCT_PUBLISH

39. Intégration Inventory

GET stock

```
-> INVENTORY_VIEW

AdjustStock
-> INVENTORY_ADJUST
-> audit actor userId/memberId

TransferStock
-> INVENTORY_TRANSFER
-> vérifier droits source + destination
```

40. Intégration Marketplace Order

```
Confirm SellerOrder
-> ORDER_CONFIRM

Mark Preparing
-> ORDER_MARK_PREPARING

Mark Ready
-> ORDER_MARK_READY

Reject
-> ORDER_REJECT

Chaque mutation stocke:
actorUserId
actorMemberId
actorStoreId
```

41. Intégration Review / Return

```
ReplyToReview
-> REVIEW_REPLY

ApproveReturn
-> RETURN_APPROVE

RejectReturn
-> RETURN_REJECT

InspectReturn
-> RETURN_INSPECT
```

42. UI HBAExpress Pro - Membres

```
Paramètres / Équipe

[ + Ajouter un membre ]

David K.
Responsable commandes
HBA Tech Store
ACTIVE
[Voir] [Modifier accès] [Suspendre]

Sophie A.
Gestionnaire stock
HBA Tech Store + Store Akpakpa
ACTIVE
[Voir] [Modifier accès] [Suspendre]
```

43. Formulaire d'invitation

```
Ajouter un membre

Email *
[ employee@example.com ]

Fonction
[ Responsable commandes ]

Boutiques *
[x] HBA Tech Store
[ ] HBA Store Akpakpa

Rôle
[ Order Manager ]

Permissions avancées
[ Personnaliser ]

[ Envoyer l'invitation ]
```

44. Écran permissions avancées

```
Produits
[x] Voir
[ ] Créer
[ ] Modifier
[ ] Publier
```

```

Commandes
[x] Voir
[x] Confirmer
[x] En préparation
[x] Prête
[ ] Annuler

Stock
[x] Voir
[ ] Ajuster
[ ] Transférer

Finance
[ ] Voir wallet
[ ] Retraits

Membres
[ ] Inviter
[ ] Modifier les rôles

```

45. API d'activité

```

GET /api/v1/seller/audit?storeId=store_001&memberId=sm_001

{
  "success": true,
  "data": [
    {
      "occurredAt": "2026-08-19T09:32:00Z",
      "actor": {
        "memberId": "sm_001",
        "displayName": "David K."
      },
      "action": "ORDER_MARKED_PREPARING",
      "resourceId": "sorder_123"
    }
  ]
}

```

46. Codes erreurs

```

SELLER_MEMBER_NOT_FOUND
SELLER_MEMBER_NOT_ACTIVE
STORE_MEMBERSHIP_NOT_FOUND
STORE_MEMBERSHIP_NOT_ACTIVE
PERMISSION_DENIED
ROLE_NOT_FOUND
ROLE_SCOPE_INVALID
ROLE_OWNER_ONLY
CANNOT_ESCALATE_PRIVILEGES
CANNOT_REMOVE_LAST_OWNER
INVITATION_NOT_FOUND
INVITATION_EXPIRED
INVITATION_ALREADY_ACCEPTED
INVITATION_REVOKED
INVITATION_EMAIL_MISMATCH
MEMBER_ALREADY_EXISTS
MEMBER_ALREADY_ASSIGNED_TO_STORE
STORE_NOT_OWNED_BY_SELLER
SENSITIVE_ACTION_REAUTH_REQUIRED
CONCURRENCY_CONFLICT

```

47. Réponse erreur standard

```

{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Vous n'avez pas l'autorisation requise.",
    "details": {
      "requiredPermission": "INVENTORY_ADJUST",
      "storeId": "store_001"
    },
    "traceId": "4bf92f..."
  }
}

```

48. Structure .NET 9 recommandée

```

HBA.SellerService/
■■■■ src/
■ ■■■■ HBA.Seller.Domain/
■ ■ ■■■■ Aggregates/
■ ■ ■ ■ ■ Seller/
■ ■ ■ ■ ■ SellerMember/
■ ■ ■ ■ ■ SellerInvitation/
■ ■ ■■■■ Entities/
■ ■ ■ ■ ■ StoreMembership.cs
■ ■ ■ ■ ■ Role.cs
■ ■ ■ ■ ■ Permission.cs
■ ■ ■■■■ ValueObjects/

```

```

■ ■ ■■■ Policies/
■ ■ ■■■ Events/
■ ■ ■■■ Repositories/
■ ■■■ HBA.Seller.Application/
■ ■ ■■■ Members/
■ ■ ■■■ InviteMember/
■ ■ ■■■ AcceptInvitation/
■ ■ ■■■ SuspendMember/
■ ■ ■■■ RevokeMember/
■ ■ ■■■ AssignStores/
■ ■ ■■■ UpdateMemberRoles/
■ ■ ■■■ Roles/
■ ■ ■■■ Authorization/
■ ■ ■■■ Audit/
■ ■ ■■■ Queries/
■ ■■■ HBA.Seller.Infrastructure/
■ ■ ■■■ Persistence/
■ ■ ■■■ Kafka/
■ ■ ■■■ Outbox/
■ ■ ■■■ Inbox/
■ ■ ■■■ Grpc/
■ ■ ■■■ Redis/
■ ■ ■■■ Security/
■ ■■■ HBA.Seller.Api/
■ ■■■ Endpoints/
■ ■■■ GrpcServices/
■ ■■■ Authorization/
■ ■■■ Program.cs
■■■ proto/
■■■ tests/
■ ■■■ UnitTests/
■ ■■■ IntegrationTests/
■ ■■■ ContractTests/
■ ■■■ E2ETests/
■■■ HBA.SellerService.sln

```

49. Authorization handler .NET

```

// intention d'architecture

[Authorize]
[RequireStorePermission(Permissions.OrderConfirm)]
POST /seller/orders/{id}/confirm

PermissionHandler:
1. récupérer userId depuis ClaimsPrincipal
2. résoudre sellerId/storeId de la ressource
3. charger EffectiveAccess
4. vérifier membre ACTIVE
5. vérifier permission
6. refuser par défaut

```

50. Cache d'accès

Les permissions effectives sont de bons candidats au cache Redis. Le cache doit être invalidé dès modification d'un membre, d'un rôle ou d'une affectation.

```

Key:
seller-access:{sellerId}:{storeId}:{userId}

Value:
{
  "memberId": "sm_001",
  "status": "ACTIVE",
  "roles": ["ORDER_MANAGER"],
  "permissions": ["ORDER_VIEW", "ORDER_CONFIRM", ...],
  "accessVersion": 12
}

TTL court + invalidation Kafka.

```

51. Tests obligatoires

- invitation valide puis acceptation
- invitation expirée refusée
- invitation révoquée refusée
- email d'acceptation différent refusé
- membre déjà existant
- membre sur plusieurs boutiques
- rôles différents selon boutique
- Store Manager ne voit pas une boutique non affectée
- Order Manager peut confirmer mais pas ajuster le stock

- Inventory Manager peut ajuster mais pas confirmer une commande
- Catalog Manager peut soumettre un produit mais pas contourner validation admin
- membre suspendu perd immédiatement l'accès
- invalidation cache après permissions-updated
- impossibilité d'escalade de privilèges
- impossibilité de supprimer le dernier Owner
- action financière critique requiert politique renforcée
- Outbox publie member.roles-updated
- consumer Kafka idempotent
- audit contient actorUserId/memberId
- concurrence sur modification des rôles

52. Scénario E2E - Order Manager

1. Owner invite david@example.com
2. Store = store_001
3. Role = ORDER_MANAGER
4. David accepte invitation
5. SellerMember ACTIVE
6. David se connecte
7. HBAExpress Pro affiche commandes store_001
8. David confirme SellerOrder
9. Order Service vérifie ORDER_CONFIRM
10. commande CONFIRMED
11. audit enregistre David
12. David tente Inventory Adjust
13. accès refusé: INVENTORY_ADJUST absent

53. Scénario E2E - suspension immédiate

1. Sophie est INVENTORY_MANAGER
2. permissions en cache Redis
3. Owner suspend Sophie
4. DB: SellerMember = SUSPENDED
5. Outbox -> seller.member.suspended
6. Kafka consumer invalide cache
7. requête suivante de Sophie
8. accès refusé

54. Scénario E2E - multi-store

User: employee_001

Store A:
ORDER_MANAGER

Store B:
INVENTORY_MANAGER

Résultat:

Store A -> peut confirmer commande
Store A -> ne peut pas ajuster stock

Store B -> peut ajuster stock
Store B -> ne peut pas confirmer commande

55. Règles non négociables pour Claude Code

- Ne jamais partager le compte Owner avec les employés.
- Identity/User Service possède l'identité ; Seller Service possède membership/RBAC.
- Un User peut appartenir à plusieurs boutiques et potentiellement plusieurs Seller organisations.
- Les droits sont évalués avec sellerId + storeId + userId.
- Refus par défaut si le contexte d'autorisation est incomplet.
- Ne jamais faire confiance au sellerId/storeId fourni par le client pour l'autorisation.
- Les permissions sensibles sont explicitement séparées.
- Le dernier Owner ne peut pas être supprimé sans transfert contrôlé.
- Un acteur ne peut pas accorder des privilèges supérieurs aux siens.

- Les invitations sont expirables, révocables et à usage unique.
- Stocker le hash du token d'invitation, pas le token brut.
- Tous les appels synchrones inter-services sont en gRPC.
- Tous les événements asynchrones sont en Kafka.
- Transactional Outbox et Inbox/idempotence obligatoires.
- Les caches d'autorisation doivent être invalidables immédiatement.
- Chaque action sensible doit être auditée avec l'acteur réel.
- Les mutations utilisent une gestion de concurrence/version.
- Écrire migrations EF Core 9, proto, OpenAPI, événements Kafka et tests.

56. Critères d'acceptation

Le module est accepté lorsqu'un Owner peut inviter des employés, les affecter à une ou plusieurs boutiques, attribuer des rôles système ou personnalisés, suspendre/révoquer les accès et consulter l'audit. Les services Marketplace doivent pouvoir déterminer de façon fiable si un membre possède une permission sur une boutique, sans escalade de privilèges et avec invalidation rapide des droits modifiés.